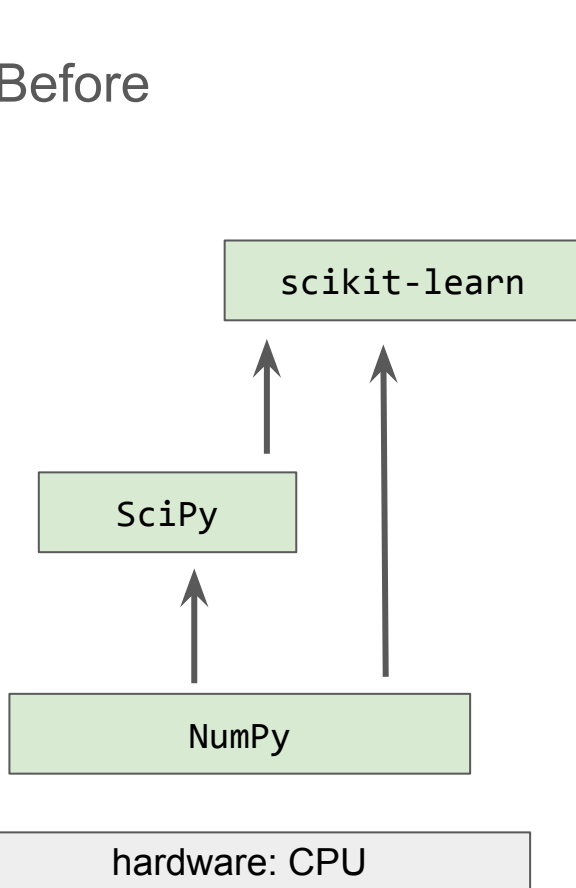


SciPy's support for GPUs, PyTorch and CuPy (and Jax)

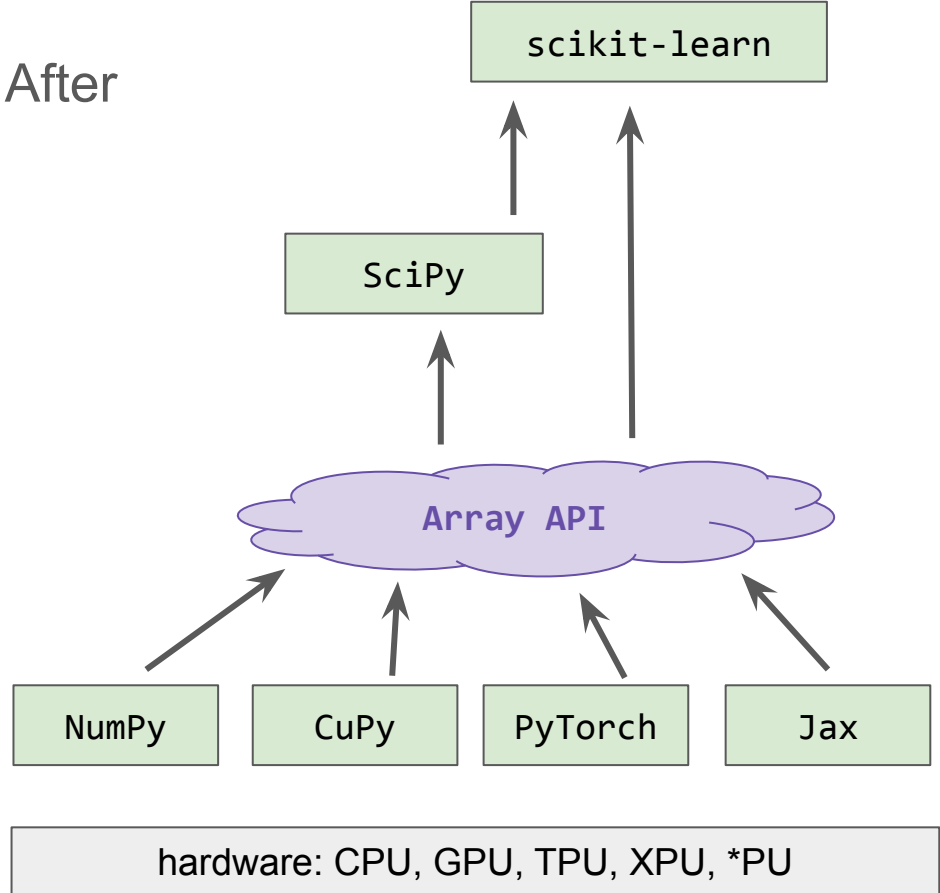
Evgeni Burovski  
Quansight

# Array API

Before



After



# Array API

**All users develop against a common API, and it just works for all array libraries across all hardware types.**

User libraries have to migrate:

[https://data-apis.org/array-api/draft/migration\\_guide.html](https://data-apis.org/array-api/draft/migration_guide.html)

[https://data-apis.org/array-api/draft/tutorial\\_basic.html](https://data-apis.org/array-api/draft/tutorial_basic.html)

[https://data-apis.org/array-api/draft/tutorial\\_api.html](https://data-apis.org/array-api/draft/tutorial_api.html)

# Array API support ecosystem

- `array-api-compat`  
(temporary) smooth over small incompatibilities
- `array-api-extra`  
useful extras
- `array-api-strict`  
testing consumer compliance
- `array-api-tests`  
testing producer compliance

SciPy vendors `array-api-compat` and `array-api-extra`.

# SciPy Array API support

As of SciPy 1.18, Array API support is experimental.  
(Mainline: planned for SciPy 2.0)

```
$ export SCIPY_ARRAY_API=1
```

SciPy supports NumPy, CuPy, PyTorch (CPU and GPU) and JAX  
(CPU and GPU).

(plus limited dask support)

# SciPy Array API coverage

## Support on CPU

| module                                   | torch | jax  | dask |
|------------------------------------------|-------|------|------|
| <a href="#">cluster.vq</a> (4)           | 100%  | 100% | 100% |
| <a href="#">cluster.hierarchy</a> (29)   | 97%   | 97%  | 97%  |
| <a href="#">constants</a> (3)            | 100%  | 100% | 100% |
| <a href="#">differentiate</a> (3)        | 100%  | 100% | 0%   |
| <a href="#">fft</a> (32)                 | 100%  | 94%  | 100% |
| <a href="#">integrate</a> (19)           | 37%   | 21%  | 26%  |
| <a href="#">interpolate</a> (14)         | 43%*  | 43%* | 43%* |
| <a href="#">io</a> (9)                   | 0%*   | 0%*  | 0%*  |
| <a href="#">linalg</a> (95)              | 3%*   | 3%*  | 2%*  |
| <a href="#">linalg.interpolative</a> (9) | 0%*   | 0%*  | 0%*  |
| <a href="#">ndimage</a> (73)             | 100%  | 100% | 100% |

## Support on GPU

| module                                   | cupy | torch | jax  |
|------------------------------------------|------|-------|------|
| <a href="#">cluster.vq</a> (4)           | 25%  | 25%   | 25%  |
| <a href="#">cluster.hierarchy</a> (29)   | 28%  | 28%   | 28%  |
| <a href="#">constants</a> (3)            | 100% | 100%  | 100% |
| <a href="#">differentiate</a> (3)        | 100% | 100%  | 100% |
| <a href="#">fft</a> (32)                 | 75%  | 75%   | 75%  |
| <a href="#">integrate</a> (19)           | 42%  | 37%   | 21%  |
| <a href="#">interpolate</a> (14)         | 0%*  | 0%*   | 0%*  |
| <a href="#">io</a> (9)                   | 0%*  | 0%*   | 0%*  |
| <a href="#">linalg</a> (95)              | 3%*  | 3%*   | 3%*  |
| <a href="#">linalg.interpolative</a> (9) | 0%*  | 0%*   | 0%*  |
| <a href="#">ndimage</a> (73)             | 93%  | 0%    | 1%   |

[https://docs.scipy.org/doc/scipy/dev/api-dev/array\\_api.html#support-on-cpu](https://docs.scipy.org/doc/scipy/dev/api-dev/array_api.html#support-on-cpu)

# SciPy Array API coverage

## Legend

✓ = supported

✗ = unsupported

N/A = out-of-scope

| function                             | torch | jax | dask |
|--------------------------------------|-------|-----|------|
| <a href="#">cubature</a>             | ✓     | ✓   | ✓    |
| <a href="#">cumulative_simpson</a>   | ✓     | ✗   | ✓    |
| <a href="#">cumulative_trapezoid</a> | ✓     | ✓   | ✓    |
| <a href="#">dblquad</a>              | ✗     | ✗   | ✗    |
| <a href="#">fixed_quad</a>           | ✗     | ✗   | ✗    |

## Array API Standard Support

[cumulative\\_simpson](#) has experimental support for Python Array API Standard compatible backends in addition to NumPy. Please consider testing these features by setting an environment variable `SCIPY_ARRAY_API=1` and providing CuPy, PyTorch, JAX, or Dask arrays as array arguments. The following combinations of backend and device (or other capability) are supported.

| Library | CPU              | GPU |
|---------|------------------|-----|
| NumPy   | ✓                | n/a |
| CuPy    | n/a              | ✓   |
| PyTorch | ✓                | ✓   |
| JAX     | ✗                | ✗   |
| Dask    | ⚠ computes graph | n/a |

See [Support for the array API standard](#) for more information.

[https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.cumulative\\_simpson.html](https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.cumulative_simpson.html)

# SciPy Array API coverage

```
@xp_capabilities(allow_dask_compute=1,
                  skip_backends=["jax.numpy", "item assignment"])]
def cumulative_simpson(y, *, x=None, dx=1.0, axis=-1, initial=None):
```

## Legend

✓ = supported

✗ = unsupported

N/A = out-of-scope

| function                             | torch | jax | dask |
|--------------------------------------|-------|-----|------|
| <a href="#">cubature</a>             | ✓     | ✓   | ✓    |
| <a href="#">cumulative_simpson</a>   | ✓     | ✗   | ✓    |
| <a href="#">cumulative_trapezoid</a> | ✓     | ✓   | ✓    |
| <a href="#">dblquad</a>              | ✗     | ✗   | ✗    |
| <a href="#">fixed_quad</a>           | ✗     | ✗   | ✗    |

## Array API Standard Support

[cumulative\\_simpson](#) has experimental support for Python Array API Standard compatible backends in addition to NumPy. Please consider testing these features by setting an environment variable `SCIPY_ARRAY_API=1` and providing CuPy, PyTorch, JAX, or Dask arrays as array arguments. The following combinations of backend and device (or other capability) are supported.

| Library | CPU              | GPU |
|---------|------------------|-----|
| NumPy   | ✓                | n/a |
| CuPy    | n/a              | ✓   |
| PyTorch | ✓                | ✓   |
| JAX     | ✗                | ✗   |
| Dask    | ⚠ computes graph | n/a |

See [Support for the array API standard](#) for more information.

[https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.cumulative\\_simpson.html](https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.cumulative_simpson.html)

# Array API “capabilities” framework

Both documentation and tests are generated from “capabilities”.

```
@xp_capabilities(allow_dask_compute=1,
                 skip_backends=(["jax.numpy", "item assignment"]))
def cumulative_simpson(y, *, x=None, dx=1.0, axis=-1, initial=None):
    ...

@make_xp_test_case(cumulative_simpson)
class TestCumulativeSimpson:
    ...
```

| Library | CPU              | GPU |
|---------|------------------|-----|
| NumPy   | ✓                | n/a |
| CuPy    | n/a              | ✓   |
| PyTorch | ✓                | ✓   |
| JAX     | ✗                | ✗   |
| Dask    | ⚠ computes graph | n/a |

See [Support for the array API standard](#) for more information.

# Testing support

- `xp_assert_close, xp_assert_equal`  
Backend-native, stricter than `np.testing`
- `default_xp`
- `@skip_xp_backends("cupy", reason="cupy#1235")`
- `lazy_xp_function`  
(from `array-api-extra.testing`) computational graphs, JIT

# Compiled code

## The sandwich pattern

```
def foo(x):  
    xp = array_namespace(x)  
    x_np = np.asarray(x)  
    result = compiled_function(x_np)  
    return xp.asarray(result)
```

- Zero-copy for CPU tensors
- Incurs a device transfer for GPU tensors (hence don't do)

# Compiled code

## Delegation

```
scipy.ndimage.sobel(cupy_array, ...) delegates to  
cupyx.scipy.ndimage.sobel
```

- Used in `scipy.ndimage`, `signal` and `interpolate`
- Delegation is namespace based.
- Currently requires exactly matching functions (`cupyx.scipy`, `jax.scipy`)
- CuPy issues/limitations propagate to SciPy users
- Under discussion: “compute” namespaces  
    make torch GPU tensors use CuPy

# Compiled code

## Multiple “backends” (internal delegation)

SciPy itself maintains two codebases: “compiled” and “generic”

- `spatial.Rotation`, `interpolate.RBFInterpolator`, `stats/special`
- Backend selection is manual  
`if is_numpy(xp): ... else ...`
- Type dispatching is under discussion (via e.g. `spatch`)

An ecosystem-wide solution vs complexity

- Maintenance load is a concern  
Have to ensure exact equivalence
- Accelerator specific code is out of question.  
SciPy cannot maintain CUDA kernels.

# AOT vs JIT

## JIT the “generic” backend code

Let `torch.compile` generate kernels for torch GPU tensors

- Public API
  - Might need to reach into private details (only `jit` an internal function)
  - JITs are not universal
  - `pytree` protocol (?)
  - `spatch TorchJITBackend, JaxJITBackend` (?)
- Performance vs manually written kernels
- Maintenance burden: AOT “numpy” backend vs “generic” backend (plus `jit/eager`)

<https://github.com/serge-sans-paille/pythran/issues/2367>

# AOT vs JIT, can unify?

Can generate from the same code

- AOT compiled kernels on CPU
- JIT compiled kernels otherwise

Pythran:

```
# pythran export foo(float[:], numpy pkg)
def foo(x, xp):
    return xp.sum(xp.sin(x**2))
```

Implemented in Pythran: <https://github.com/serge-sans-paille/pythran/issues/2367>

SciPy POC (not mergeable): <https://github.com/scipy/scipy/pull/24306>

# Conclusions and outlook

- SciPy supports CuPy, torch, jax on CPU and GPU via Array API
  - Developed/dogfooded useful Array API support infrastructure
  - Graduating the support from experimental to mainline: “soon” (tm)
- 
- How to ~~not-explode~~ minimize the maintenance burden
  - How to consolidate best practices across the ecosystem
  - How to deal with AOT vs JIT compiled code